

Serwery poczty w aplikacjach web

Wykłady 11

Bartosz Lauks

18 maja 2026



- ① Role e-maili w aplikacjach web
- ② Budowa wiadomości e-mail (nagłówki, body, MIME)
- ③ Za kulisami działania poczty elektronicznej
- ④ Implementacja systemu pocztowego w aplikacji web

- 1 Role e-maili w aplikacjach web
- 2 Budowa wiadomości e-mail (nagłówki, body, MIME)
- 3 Za kulisami działania poczty elektronicznej
- 4 Implementacja systemu pocztowego w aplikacji web

Role e-maili w aplikacjach web

Rola e-maili w aplikacjach web

- **E-mail (ang. electronic mail)** to jedna z najstarszych, a zarazem najbardziej uniwersalnych form komunikacji w Internecie.
- Pomimo rozwoju nowych kanałów kontaktu, takich jak powiadomienia push, SMS czy komunikatory, e-mail pozostaje podstawowym narzędziem do oficjalnej i systemowej komunikacji w aplikacjach webowych.

Typowe zastosowania e-maili w aplikacjach web

- **Potwierdzenie rejestracji i aktywacja konta** - Po utworzeniu konta użytkownik otrzymuje wiadomość z linkiem aktywacyjnym.
- **Resetowanie hasła** - użytkownik otrzymuje jednorazowy link umożliwiający bezpieczne ustawienie nowego hasła.
- **Powiadomienia systemowe** - Informacje o zmianach statusu zamówienia, nowych wiadomościach, komentarzach, aktualizacjach konta itp.
- **Newslettery i e-mail marketing** - Regularne wiadomości z nowościami, promocjami, ofertami.
- **Wiadomości transakcyjne** - Potwierdzenia zakupu, faktury, raporty PDF generowane dynamicznie.

Role e-maili w aplikacjach web

Dlaczego e-mail jest nadal tak istotny?

- **Powszechność** - większość usług internetowych wymaga adresu e-mail.
- **Uniwersalność** - E-mail działa niezależnie od platformy, systemu operacyjnego, aplikacji.
- **Asynchroniczność** - Wiadomości trafiają do odbiorcy, nawet jeśli ten nie jest online. E-mail pozwala na przekazywanie danych, które użytkownik może zachować.
- **Automatyzacja** - Możliwość łatwego tworzenia reguł, szablonów, harmonogramów wysyłki.

- 1 Role e-maili w aplikacjach web
- 2 Budowa wiadomości e-mail (nagłówki, body, MIME)
- 3 Za kulisami działania poczty elektronicznej
- 4 Implementacja systemu pocztowego w aplikacji web

Budowa wiadomości e-mail (nagłówki, body, MIME)

Ogólna struktura wiadomości e-mail

- Standardowa wiadomość e-mail, zgodnie z protokołem SMTP (RFC 5322), składa się z dwóch głównych części:
 - ➊ **Nagłówki (Headers)** – zawierają metadane: od kogo, do kogo, temat, datę, itp.
 - ➋ **Treść wiadomości (Body)** – właściwa zawartość, którą widzi użytkownik (tekst, HTML, załączniki itp.)
- Format wiadomości tekstowej to zwykły plik tekstowy, gdzie:
 - Poszczególne nagłówki oddzielone są od siebie znakami nowej linii.
 - Po nagłówkach następuje pusta linia, która oddziela nagłówki od treści.
 - Treść wiadomości zaczyna się po tej pustej linii.

Budowa wiadomości e-mail (nagłówki, body, MIME)

Nagłówki wiadomości e-mail

- Nagłówki to linie tekstowe w formacie Nazwa: Wartość. Są przetwarzane przez serwery pocztowe i klientów pocztowych. Najczęściej używane nagłówki:
 - **From** - Nadawca wiadomości (adres e-mail)
 - **To** - Główny odbiorca wiadomości
 - **Cc** - Kopia wiadomości do innych odbiorców (Carbon Copy)
 - **Bcc** - Ukryta kopia wiadomości (Blind Carbon Copy) – nie widoczna dla innych
 - **Subject** - Temat wiadomości
 - **Date** - Data i godzina nadania (np. Mon, 20 May 2024 14:23:10 +0200)
 - **Message-ID** - Unikalny identyfikator wiadomości (generowany przez serwer nadawcy)
 - **Reply-To** - Adres, na który ma być wysłana odpowiedź
 - **Return-Path** - Adres zwrotny do obsługi błędów dostarczenia
 - **Content-Type** - Typ zawartości (tekst, HTML, załączniki) – kluczowy dla MIME
 - **MIME-Version** - Wersja standardu MIME (zazwyczaj: 1.0)

Budowa wiadomości e-mail (nagłówki, body, MIME)

Czym jest MIME

- **MIME (ang. Multipurpose Internet Mail Extensions)** to standard internetowy, który rozszerza format wiadomości poczty elektronicznej, umożliwiając przesyłanie:
 - tekstu w różnych kodowaniach (np. UTF-8),
 - załączników (pliki graficzne, wideo, audio, PDF itp.),
 - wiadomości w formacie HTML,
 - wielu części wiadomości (multipart), np. tekst + załączniki.
- MIME definiuje:
 - typy danych (**Content-Type**), np. `text/plain`, `text/html`, `image/jpeg`, `application/pdf`,
 - kodowanie treści (**Content-Transfer-Encoding**), np. `base64`, `quoted-printable`,
 - struktury wieloczęściowe (**multipart**), np. `multipart/mixed`, `multipart/alternative`.

Serwery poczty w aplikacjach web

Protokoły poczty elektronicznej

- Wysyłanie i odbieranie wiadomości e-mail opiera się na zestawie ustandaryzowanych protokołów. Dzielią się one na dwie główne grupy:
 - **Protokoły wysyłania wiadomości – np. SMTP**
 - **Protokoły odbierania wiadomości – np. POP3, IMAP**
- Każdy z tych protokołów pełni inną funkcję, działa na innych portach, a także ma swoje zalety i ograniczenia.

Serwery poczty w aplikacjach web

SMTP

- **SMTP (Simple Mail Transfer Protocol)** to protokół wysyłania wiadomości e-mail.
- Jest wykorzystywany przez aplikacje webowe, serwery pocztowe i klientów e-mail do przesyłania wiadomości do serwera odbiorcy.
- **Działanie:**
 - Aplikacja łączy się z serwerem SMTP (np. **smtp.gmail.com**).
 - Przekazuje nadawcę (**MAIL FROM**) i odbiorcę (**RCPT TO**).
 - Przesyła treść wiadomości (**DATA**).
 - Serwer przekazuje wiadomość do kolejnych serwerów lub zapisuje ją lokalnie.
- **Porty:**
 - **25** - SMTP bez szyfrowania (czasem blokowany)
 - **587** - SMTP z START TLS (zalecany)
 - **465** - SMTP z SSL/TLS

Serwery poczty w aplikacjach web

POP3

- **POP3 (Post Office Protocol v3)** to protokół odbierania wiadomości, służący do pobierania e-maili z serwera do klienta.
- Jego działanie przypomina tradycyjną skrzynkę pocztową. Po pobraniu wiadomości są usuwane z serwera (domyślnie).
- **Cechy:**
 - Prosty i lekki protokół.
 - Nadaje się do połączeń z jednego urządzenia.
 - Nie obsługuje synchronizacji stanu folderów, znaczników itp.
- **Porty:**
 - **110** - POP3 bez szyfrowania
 - **995** - POP3 z SSL/TLS

Serwery poczty w aplikacjach web

IMAP

- **IMAP (Internet Message Access Protocol)** to nowocześniejszy i bardziej zaawansowany protokół odbierania wiadomości, który pozwala na synchronizację e-maili między wieloma urządzeniami.
- **Cechy:**
 - Wiadomości pozostają na serwerze.
 - Obsługuje foldery, oznaczanie wiadomości (np. jako przeczytane).
 - Idealny do korzystania z tej samej skrzynki na wielu urządzeniach.
- **Porty:**
 - **143** - IMAP bez szyfrowania
 - **993** - IMAP z SSL/TLS

Serwery poczty w aplikacjach web

DNS a e-mail

- Na pierwszych zajęciach poznaliśmy główne zastosowanie (**Domain Name System**), który tłumaczy nazwy domenowe (np. **example.com**) na adresy **IP** (np. **192.0.2.1**) i odwrotnie.
- DNS udostępnia dodatkowo rekordy, które między innymi określają:
 - gdzie kierować ruch internetowy (www, e-mail, ftp),
 - kto obsługuje pocztę dla domeny,
 - jakie są zabezpieczenia e-mail (**SPF, DKIM, DMARC**),

Serwery poczty w aplikacjach web

Wpis MX (Mail Exchange Record)

- **Rekord MX** określa, które serwery są odpowiedzialne za przyjmowanie poczty e-mail dla danej domeny.
- Gdy ktoś wysyła wiadomość e-mail do adresata w domenie **example.com**, serwer nadawcy zapytuje **DNS o rekord MX** dla tej domeny.
- Odpowiedź zawiera jeden lub więcej rekordów wskazujących nazwę hosta serwera pocztowego (np. **mail.example.com**) oraz jego priorytet.

Serwery poczty w aplikacjach web

Wpis PTR (Pointer Record, Reverse DNS)

- **Rekord PTR** służy do tłumaczenia adresu **IP** na nazwę domenową. Jest to odwrotność rekordu **A**. Nazywany jest również rekordem **reverse DNS**.
- Serwery pocztowe używają odwrotnego DNS (rDNS) do weryfikacji autentyczności i wiarygodności serwera nadawcy.
- Jeśli serwer SMTP odbierający wiadomość otrzyma połączenie z **IP 192.0.2.10**, może wykonać zapytanie **typu PTR**, aby sprawdzić, jaka nazwa domenowa jest z nim powiązana.

Serwery poczty w aplikacjach web

SPF (Sender Policy Framework)

- **SPF** to mechanizm weryfikujący, czy serwer wysyłający wiadomość e-mail jest uprawniony do wysyłania poczty w imieniu danej domeny.
- Nadawca (np. **user@example.com**) wysła wiadomość z serwera o **IP 192.0.2.10**.
- Serwer odbiorczy sprawdza rekord SPF w DNS domeny example.com
- Rekord SPF zawiera listę dozwolonych adresów IP i hostów, które mogą wysyłać wiadomości z tej domeny.

Serwery poczty w aplikacjach web

DKIM (DomainKeys Identified Mail)

- **DKIM** to mechanizm umożliwiający podpisywanie wiadomości e-mail kluczem kryptograficznym, co pozwala odbiorcy zweryfikować, że wiadomość nie została zmodyfikowana w trakcie transmisji i rzeczywiście pochodzi z danej domeny.
- Serwer nadawcy dodaje do nagłówka wiadomości podpis cyfrowy (kluczem prywatnym).
- Serwer odbiorczy pobiera klucz publiczny z DNS domeny nadawcy i sprawdza podpis.
- Jeśli podpis jest poprawny → wiadomość uznana za autentyczną.

Serwery poczty w aplikacjach web

DMARC (Domain-based Message Authentication, Reporting & Conformance)

- **DMARC** to mechanizm wykorzystujący **SPF i DKIM**, który pozwala właścicielowi domeny określić, jak odbiorcy mają postępować z wiadomościami, które nie przejdą weryfikacji SPF i/lub DKIM.
- Serwer odbiorczy sprawdza, czy wiadomość:
 - przeszła SPF lub DKIM,
 - oraz czy domena użyta w nagłówku "From" zgadza się z domeną z SPF/DKIM (tzw. "alignment").

- 1 Role e-maili w aplikacjach web
- 2 Budowa wiadomości e-mail (nagłówki, body, MIME)
- 3 Za kulisami działania poczty elektronicznej
- 4 Implementacja systemu pocztowego w aplikacji web

Za kulisami działania poczty elektronicznej

1. Wysyłanie e-maila z lokalnej aplikacji lub skryptu

- Użytkownik tworzy wiadomość e-mail.
- Wiadomość zawiera nagłówki (**From, To, Subject, MIME-Type**) i treść (**plaintext** lub **HTML**).
- Aplikacja kontaktuje się z lokalnym serwerem SMTP, np. skonfigurowanym w XAMPP (np. Mercury Mail lub Sendmail dla Windows/Linux).

2. Dostarczanie wiadomości do lokalnego serwera SMTP

- Aplikacja przekazuje e-mail lokalnemu serwerowi SMTP (np. na [localhost:25](#)).
- Serwer SMTP analizuje adres docelowy (recipient@example.com) i rozpoczyna proces dostarczania wiadomości.

Za kulisami działania poczty elektronicznej

3. Sprawdzenie adresu docelowego (DNS lookup)

- Serwer **SMTP** wykonuje zapytanie **DNS**, aby dowiedzieć się, gdzie należy wysłać e-mail.
- Używany jest rekord **MX (Mail eXchanger)** dla domeny odbiorcy

4. Nawiązywanie połączenia SMTP z serwerem docelowym

- Lokalny serwer **SMTP** łączy się z zewnętrznym serwerem pocztowym odbiorcy (np. **mx1.example.com**) na porcie **25**.
- Przeprowadzana jest wymiana danymi przez SMTP

Za kulisami działania poczty elektronicznej

5. Sprawdzenia bezpieczeństwa i autoryzacji

- Serwer odbiorczy może wykonać różne testy, aby zaakceptować lub odrzucić wiadomość:
 - **SPF (Sender Policy Framework)** – sprawdza, czy IP serwera nadawcy ma prawo wysyłać e-maile w imieniu domeny nadawcy.
 - **DKIM (DomainKeys Identified Mail)** – weryfikuje podpis kryptograficzny w nagłówku wiadomości.
 - **DMARC** – mówi serwerowi, jak postępować w przypadku niezgodności SPF lub DKIM.
 - **Reverse DNS (PTR)** – sprawdza, czy IP serwera SMTP ma przypisany prawidłowy DNS.
 - **Blacklisty (RBL)** – sprawdzenie, czy serwer nadawcy nie znajduje się na czarnej liście.
- Jeśli któryś z testów zakończy się niepowodzeniem, wiadomość może trafić do spamu lub zostać odrzucona.

Za kulisami działania poczty elektronicznej

6. Dostarczenie wiadomości do skrzynki odbiorczej

- Jeśli wszystko przebiegnie pomyślnie wiadomość trafia do kolejki przychodzącej serwera odbiorczego.
- System pocztowy zapisuje e-mail w skrzynce użytkownika.

7. Odebranie e-maila przez użytkownika

- Klient łączy się z serwerem przychodzącym:
 - **POP3** – pobiera e-maile lokalnie (czasem usuwa z serwera).
 - **IMAP** – synchronizuje e-maile z serwerem (wiadomości zostają na serwerze).

Za kulisami działania poczty elektronicznej

Co jest potrzebne do lokalnego serwera poczty (SMTP)?

- **Oprogramowanie serwera SMTP** - (np. Postfix (Linux), Exim, Sendmail, hMailServer (Windows)).
- **Serwer DNS z prawidłowym wpisem MX i PTR** – do rozpoznawania Twojej domeny.
- **Stały adres IP** – dynamiczne IP (jak z sieci domowej) są zwykle na czarnych listach.
- **Odpowiednie rekordy DNS: SPF DKIM DMARC** - jeśli chcesz działać publicznie.
- **Certyfikat TLS/SSL** – dla bezpiecznego połączenia.
- **Otwarte porty (np. 25, 465, 587)** – wiele dostawców Internetu blokuje port 25.

- ① Role e-maili w aplikacjach web
- ② Budowa wiadomości e-mail (nagłówki, body, MIME)
- ③ Za kulisami działania poczty elektronicznej
- ④ Implementacja systemu pocztowego w aplikacji web

Implementacja systemu pocztowego w aplikacji web

Wysyłane e-mail przy pomocy SMTP Gmail

① Skonfiguruj Gmail:

- Zaloguj się na swoje konto Gmail.
- Włącz 2FA (uwierzytelnianie dwuskładnikowe).
- Utwórz hasło aplikacji w panelu Google. To hasło aplikacji wpiszesz zamiast zwykłego hasła w kodzie.

② Zainstaluj PHPMailer:

- Pobierz PHPMailer ze strony: <https://github.com/PHPMailer/PHPMailer>.
- Skopiuj folder `src` do swojego katalogu projektu w XAMPP.
- Załaduj klasy przy pomocy `require`

③ Upewnij się, że XAMPP ma dostęp do Internetu. XAMPP lokalnie musi mieć możliwość łączenia się z zewnętrznym serwerem SMTP (`smtp.gmail.com` na porcie `465` lub `587`).

<https://localhost/lecture11/email/>

Implementacja systemu pocztowego w aplikacji web

Załączniki w e-mail

- Aplikacje WWW często tworzą i wysyłają za pomocą serwera SMTP pliki dołączone do wiadomości e-mail.
- Mogą to być np. faktury w formacie PDF lub pliki Excel, automatycznie generowane przez system w celu poinformowania innego działu firmy o dzisiejszych zamówieniach.
- Dodanie załącznika przy pomocy biblioteki **PHPMailer** jest bardzo proste. Wystarczy użyć metody `addAttachment('ścieżka/do/pliku')` na obiekcie klasy **PHPMailer**, aby dołączyć plik do wiadomości.
- Metodę `addAttachment()` można wywołać kilkakrotnie w celu dodania wielu załączników do jednej wiadomości.

localhost/lecture11/email/emailAttachment.php

Implementacja systemu pocztowego w aplikacji web

E-mail w formie HTML (MIME)

- Domyślnie zawartość wiadomości e-mail to zwykły tekst (**text/plain**), ale dzięki **MIME** możemy zadeklarować, że wiadomość ma być interpretowana jako strona internetowa.
- Dzięki temu klient pocztowy może odczytać kod **HTML i CSS**, a następnie wyświetlić wiadomość w formie przypominającej stronę WWW.
- W PHP wystarczy wywołać metodę **isHTML(true)** na obiekcie klasy **PHPMailer**, aby wiadomość została wysłana jako **HTML**.
- **Uwaga!** Nie wszystkie programy pocztowe obsługują interpretację kodu **HTML**. Niektóre mogą wyświetlić wiadomość jako zwykły tekst lub zablokować część zawartości.

localhost/lecture11/email/emailMIME.php

Implementacja systemu pocztowego w aplikacji web

Rejestracja potwierdzenie adresu email

- Najczęściej spotykaną funkcjonalnością powiązaną z pocztą w aplikacjach WWW jest potwierdzenie adresu e-mail podanego podczas rejestracji.
- Zazwyczaj działa to w następujący sposób:
 - Użytkownik wypełnia formularz rejestracyjny.
 - Dane trafiają do bazy danych, ale konto otrzymuje status „**nieaktywne**”.
 - Na adres e-mail użytkownika wysyłany jest link z tokenem aktywacyjnym.
 - Kliknięcie w link aktywuje konto.
 - Logowanie jest możliwe dopiero po aktywacji konta.

Implementacja systemu pocztowego w aplikacji web

Rejestracja potwierdzenie adresu email cd.

- Aby taka forma rejestracji była możliwa w aplikacji rozwijanej podczas wykładów, należy w pierwszej kolejności dodać do tabeli **user** następujące kolumny:
 - **activation_token** (**varchar(255)**) – będzie on dodawany do adresu URL w celu potwierdzenia autentyczności linku.
 - **is_active** (**tinyint(1)**, **Default = 0**) – dzięki niemu aplikacja będzie mogła sprawdzić, czy adres e-mail został potwierdzony i czy użytkownik może się zalogować.

Implementacja systemu pocztowego w aplikacji web

Rejestracja potwierdzenie adresu email cd.

- W skrypcie **registerServer.php** należy dodać logikę, która wyśle na adres e-mail użytkownika wiadomość zawierającą link weryfikacyjny.
- Należy utworzyć plik **activate.php**, do którego będzie prowadził link. Po przesłaniu poprawnego tokenu konto użytkownika zostanie aktywowane.
- Ostatnim krokiem jest modyfikacja pliku **loginPDO.php** w taki sposób, aby logowanie użytkownika z nieaktywnym kontem było niemożliwe.

Zmiany <http://localhost/lecture11/registerServer.php>

Zmiany <http://localhost/lecture11/loginPDO.php>

Nowy plik <http://localhost/lecture11/activate.php>

Dziękuję za uwagę !

Bartosz Lauks